MGA: Memory-Driven GUI Agent for Observation-Centric Interaction

Weihua Cheng
Shanghai Tech University
Shanghai, China
chengwh2024@shanghaitech.edu.cn

Junming Liu
Tongji University
Shanghai, China
2331890@tongji.edu.cn

Yifei Sun
East China University of Science and Technology
Shanghai, China
y21220030@mail.ecust.edu.cn

Botian Shi
Shanghai AI Laboratory
Shanghai, China
shibotian@pjlab.org.cn

Yirong Chen*
Shanghai AI Laboratory
Shanghai, China
chenyirong@pjlab.org.cn

Ding Wang†
Shanghai AI Laboratory
Shanghai, China
wangding@pjlab.org.cn

Abstract

Multimodal Large Language Models (MLLMs) have significantly advanced GUI agents, yet long-horizon automation remains constrained by two critical bottlenecks: context overload from raw sequential trajectory dependence and architectural redundancy from over-engineered expert modules. Prevailing End-to-End and Multi-Agent paradigms struggle with error cascades caused by concatenated visual-textual histories and incur high inference latency due to redundant expert components, limiting their practical deployment. To address these issues, we propose the Memory-Driven GUI Agent (MGA), a minimalist framework that decouples long-horizon trajectories into independent decision steps linked by a structured state memory. MGA operates on an "Observe First and Memory Enhancement" principle, powered by two tightly coupled core mechanisms: (1) an Observer module that acts as a task-agnostic, intent-free screen state reader to eliminate confirmation bias, visual hallucinations, and perception bias at the root; and (2) a Structured Memory mechanism that distills, validates, and compresses each interaction step into verified state deltas, constructing a lightweight state transition chain to avoid irrelevant historical interference and system redundancy. By replacing raw historical aggregation with compact, fact-based memory transitions, MGA drastically reduces cognitive overhead and system complexity. Extensive experiments on OSWorld and real-world applications demonstrate that MGA achieves highly competitive performance in open-ended GUI tasks while maintaining architectural simplicity, offering a scalable and efficient blueprint for next-generation GUI automation. https://github.com/MintyCo0kie/MGA4OSWorld.

*Corresponding author.
†Corresponding author.

Conference acronym 'XX, Woodstock, NY

ACM ISBN 978-1-4503-XXXX-X/2018/06
https://doi.org/XXXXXXX.XXXXXXX

CCS Concepts

• **Computing methodologies** → **Intelligent agents**; *Natural language processing*; *Computer vision*; Planning and scheduling; • **Human-centered computing** → *Graphical user interfaces.*

Keywords

GUI automation, multimodal large language models, structured memory, long-horizon planning, observer-first perception, active error correction

ACM Reference Format:
Weihua Cheng, Junming Liu, Yifei Sun, Botian Shi, Yirong Chen, and Ding Wang. 2026. MGA: Memory-Driven GUI Agent for Observation-Centric Interaction. In *Proceedings of Make sure to enter the correct conference title from your rights confirmation email (Conference acronym 'XX)*. ACM, New York, NY, USA, 15 pages. https://doi.org/XXXXXXX.XXXXXXX

1 Introduction

Driven by the rapid advancement of Multimodal Large Language Models (MLLMs) and agent frameworks, Graphical User Interface (GUI) agents have evolved from rule-driven approaches to complex systems capable of autonomous, human-like perception and interaction [1, 5, 6, 31, 43, 44]. However, long-horizon GUI tasks remain highly challenging due to the partial observability and dynamic layouts inherent to the GUI environment [1, 23, 45]. Existing approaches primarily fall into two paradigms: End-to-End (E2E) models and Modular Multi-Agent Systems (MAS). E2E models, such as UI-TARS [26] and Claude-Sonnet [3], directly map raw screenshots to pixel-level actions, but struggle with accumulating historical context. In contrast, MAS frameworks decompose functionality into specialized agents for planning, grounding, and execution [8, 22, 30, 31], improving modularity at the cost of increased coordination overhead.

Despite these advances, existing paradigms for long-horizon GUI interaction [2, 17, 37] still face two fundamental bottlenecks that limit their practical deployment: 1) Mainstream approaches model GUI task execution purely as flat sequential trajectories [19, 28], concatenating historical screenshots, plans, and actions into the context and requiring the model to rely on the entire raw trajectory to infer the current state (Fig. 1). This design leaves the model susceptible to misguidance from irrelevant or erroneous history, impairing its ability to track the current state. 2) To operate in

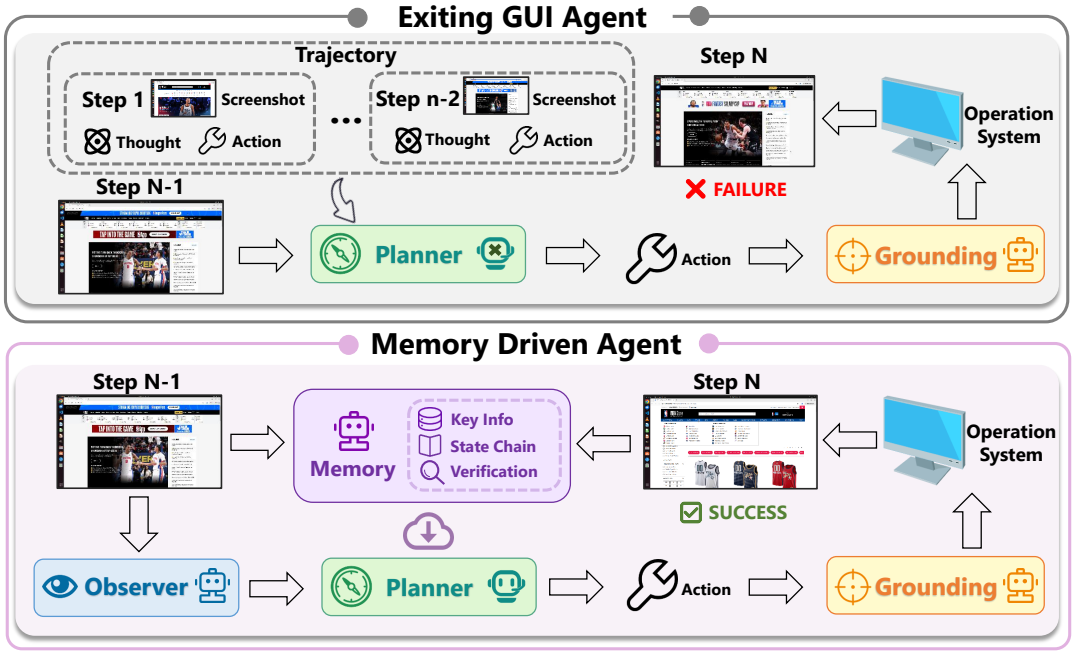


Figure 1: Comparative illustration of GUI agent workflows. Top: Conventional paradigm aggregates historical screenshots, decision chains, and action trajectories into a single context, leading to attention dilution and error accumulation. Bottom: MGA decouples redundant trajectories via structured memory, enabling a streamlined pipeline that preserves focused attention with observation-first for long-horizon tasks.

open-ended environments, state-of-the-art frameworks increasingly rely on large-scale, fully-featured expert agents. For example, OS-Symphony and Agent S3 [13, 39] integrate multiple MLLMs with orchestrators, reflection modules, and tool chains. While seemingly effective, such designs introduce redundancy and complexity, constituting over-design for the majority of routine GUI tasks.

To address the above two core bottlenecks, we revisit GUI automation from first principles and advocate reconceptualizing the core reasoning unit of GUI agents. We argue that the fusion of observation and memory forms a sufficient representation of the latent GUI state. We propose the Memory-Driven GUI Agent (MGA), which provides a novel solution to the aforementioned limitations: historical interference from raw trajectory dependence and redundancy caused by over-complex system design. MGA is built upon two tightly coupled core modules: ***Observer*** and ***Memory***.

Specifically,to eliminate perception bias at its root, we introduce the Observer module as a task-agnostic, intent-free screen state reader. Existing GUI agents typically perform perception under the guidance of task intent, leading to confirmation bias where models over-focus on actionable UI elements while neglecting critical contextual information such as textual descriptions and background states [10, 33]. This coupling between perception and decision-making produces incomplete and biased state representations, which prior work attempts to compensate for by adding complex expert modules. In contrast, the Observer in MGA is strictly prohibited from any planning or intent inference. It operates exclusively in a task-agnostic perception mode and performs only objective and fine-grained state characterization. This constraint, which enforces clarification of current factual states before accessing historical and task information, fundamentally eliminates visual hallucinations caused by subjective model inference and preserves crucial background semantics. It thus provides a pure and unbiased initial state benchmark for downstream decision-making modules, as well as a reliable input source for the paired memory mechanism.

Besides, the Memory module operates in tight coordination with the Observer to achieve sufficient GUI state representation. Rather than retaining unprocessed raw interaction trajectories, it distills, validates, and compresses each step's action and execution effect into verified state increments to construct a lightweight, append-only state transition chain decoupling long-horizon interaction trajectories into independent decision steps connected via state memory. By eliminating redundant historical burdens and interference from irrelevant or erroneous history, memory enabling reliable backtracking for the planner also retains essential contextual information including task constraints, critical verified results, and loop-monitoring anomaly signals throughout execution to avoid system redundancy and complexity and mitigate perception bias at its source, thus ensuring the memory remains compact, dependable, and efficiently accessible for real time decision-making.

In summary, this work makes the following core contributions:

- We reframe long-horizon GUI automation as independent yet logically connected decision nodes, effectively alleviating context overload caused by raw historical sequences.
- We propose MGA, an "observation first + memory enhancement" GUI agent. It leverages a task-agnostic observer to

reduce visual hallucinations, and a Structured Memory mechanism to prevent behavioral stagnation and error cascades.
- Experiments on OSWorld demonstrate that MGA maintains highly competitive performance in open-ended GUI tasks .

2 Related Work

2.1 End-to-End and Specialist GUI Models

The end-to-end paradigm integrates perception, reasoning, and execution into a unified architecture, directly mapping screen observations and user instructions to executable actions. This paradigm emphasizes model coherence, training efficiency, and cross-domain generalization [15, 20, 38, 42]. Recent studies further explore temporal dynamics and multimodal interaction for GUI agents. ScaleTrack [16] reconstructs historical trajectories and predicts future actions to capture the temporal evolution of GUI state-action dependencies. UITron-Speech [14] introduces the first speech-driven GUI agent, addressing challenges in multimodal instruction understanding and visual grounding. Representative domain-specialist models include the UI-TARS series [26], which pursues unified action modeling across applications. UI-TARS incorporates System-2 reasoning and explicit deliberation, while its follow-up UI-TARS-2 further integrates multi-round reinforcement learning and file system access for hybrid desktop/OS environments. The OpenCUA series [32] advances this line via large-scale specialist fine-tuning, with 32B and 72B variants achieving strong grounding and navigation performance across diverse OS domains. DeepMiner-Mano-7B [12] explores compact, parameter-efficient specialist architectures, balancing model scale and precise UI control. Besides task-specialized models, generalist vision-language models (VLMs) and large language models (LLMs) have also been widely adapted for GUI tasks. The Qwen3-VL-32B family [4] provides both standard instruction-following and explicit chain-of-thought reasoning variants, leveraging advanced visual understanding for screen interpretation. Claude-Sonnet-4.5 [3] and O3 [24] represent state-of-the-art generalist backbones, employing long-context modeling and chain-of-thought to perform zero/few-shot GUI control. Although these generalist models deliver strong baseline reasoning, their performance in complex dynamic environments is still limited by insufficient grounding precision and strict step-budget constraints.

2.2 Multi-Agent and Orchestrated Frameworks

Multi-agent paradigms outperform monolithic approaches in long-horizon tasks by leveraging modular decomposition and collaboration [1, 8, 18, 27, 34, 35, 46, 47]. Representative works are as follows. CoAct-1 [30] introduces a dynamic orchestrator to allocate tasks between a GUI operator and a programmer agent, enabling hybrid graphical/code execution and achieving SOTA on OSWorld. Agent S3 [13] enhances multi-agent coordination with refined global planning and constraint handling, improving success rates in multi-application workflows. UiPath [9] a hierarchical ReAct-style agent that employs an LLM-based planner to reason about task goals and generate action sequences, a visual grounder combining UI-TARS-7B with UiPath's computer vision model to map abstract actions to precise screen coordinates, and task/action reviewer components that close the execution feedback loop to trigger dynamic replanning when needed. GTA-1 [40] uses step-wise action sampling and a multimodal LLM judge to select optimal trajectories, improving robustness in complex GUI scenarios. Jedi-7B optimizes step-wise decision-making via lightweight agentic fine-tuning, achieving efficient action selection with lower computational overhead.

3 Methodology

In this work, we introduce MGA, characterizing GUI interaction tasks as a closed loop dynamic system operating in discrete time steps. As illustrated in Fig. 2, MGA integrates four cascaded, mutually complementary components that form a complete perception decision execution memory cycle: an Observation Agent, a Memory Agent, a Planning Agent, and a Grounding Agent.These components collaborate within a framework of clear data flows and functional boundaries, achieving unbiased perception, reliable memory, coherent planning, and accurate execution. The detailed prompts used in our work are shown in Appendix D.

3.1 Problem Formulation

Formally, we define the environment state at discrete time step t as a tuple that integrates current visual input, structured observation, and historical validated memory, capturing all critical information required for the agent's decision making:

$$E_t = \left(I_t,\ Z_t,\ M_{t-1}\right), \tag{1}$$

where: I_t denotes the raw screenshot of the GUI interface at step t, serving as the fundamental input for perception; Z_t is the task agnostic spatial–semantic representation extracted by the Observation Agent from I_t (detailed in Section 3.2); M_{t-1} is the memory state distilled by the Memory Agent at step $t-1$ (detailed in Section 3.3).

3.2 Observation Agent

Although many existing studies [30, 40] feed the raw screenshot I_t to the Planning Agent, its internal attention mechanism is inherently decision driven, meaning it is heavily biased toward locating actionable interface elements to generate immediate actions. However, in complex GUI tasks, non-actionable information, such as dense text descriptions, read-only data content, or system state feedback, is indispensable for comprehending and verifying task objectives [18, 29, 41]. Relying solely on the planner to process raw screenshots inevitably leads to the neglect of this critical background semantics due to "action bias." Therefore, we introduce the Observation Agent as an independent, task agnostic perception module. Its core objective is to decouple perception from decision making.

By exhaustively extracting all visual semantic content, including pure text descriptions and topological structures on the screen, it provides the planner with a complete, unbiased global state reference, fundamentally mitigating the planner's inherent blind spots in information extraction.

Specifically, at time step t, given a GUI screenshot $I_t \in \mathcal{I}$, the observation function O leverages a vision language model to map the visual input to a state representation Z_t:

$$Z_t = O(I_t) = \langle W_t, T_t, S_t \rangle, \tag{2}$$

where the observation Z_t is explicitly decoupled into three complementary dimensions:

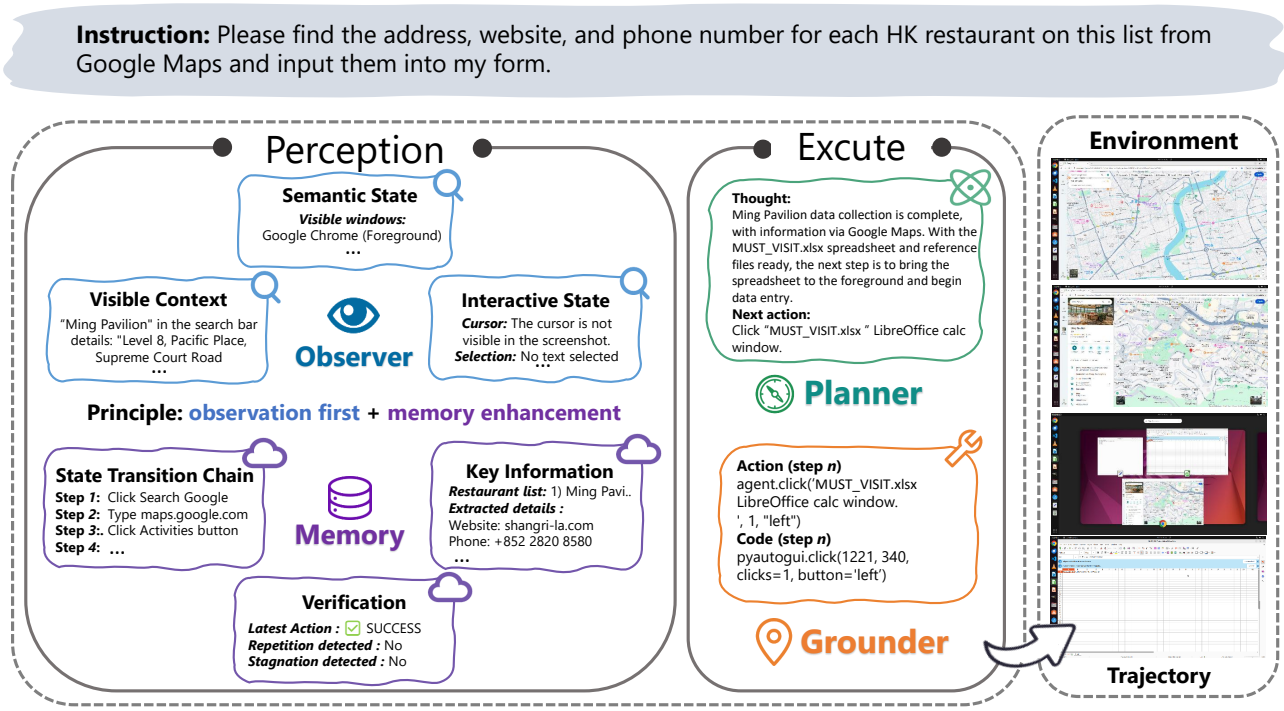


Figure 2: Detailed workflow of the proposed MGA framework on a representative *workflow* domain task within OSWorld, illustrating the internal data flow among the Observer, Memory Agent, Planner, and Grounding pipeline.

Application and Window Topology (W_t): To determine the actual actionable area of the current system, we extract the global window structure of the interface. The window topology is defined as a set of active window states:

$$W_t = \{(v_i, \phi_i, \mu_i)\}_{i=1}^{N_w}, \tag{3}$$

where v_i represents the semantic identifier of the i-th window, $\phi_i \in \{0, 1\}$ is a boolean indicator denoting whether the window is currently in the foreground and has focus, and μ_i characterizes structural constraints such as modal dialogs or system-level overlays that block interaction.

Visible Semantic Content (T_t): To capture critical non-actionable descriptions and construct a complete semantic context, we extract an exhaustive inventory of all visible text on the screen along with their corresponding UI functional attributes. The semantic content is formulated as:

$$T_t = \{(x_j, c_j)\}_{j=1}^{N_t}, \tag{4}$$

where x_j is the extracted text sequence, and $c_j \in C$ represents the UI functional category to which the text belongs. This step strictly records the global semantic information of the interface without being constrained by current short-term operational goals.

Dynamic Interaction State (S_t): Since GUIs involve state transitions and transient responses, we record the dynamic interactive attributes of the interface through a state tuple:

$$S_t = \langle \kappa_t, \sigma_t, \rho_t, \tau_t \rangle, \tag{5}$$

where κ_t denotes the current element holding keyboard focus, σ_t represents the currently selected text or regional range, ρ_t encodes the relative scroll position of the current view within the global document, and τ_t captures transient temporal overlays. This dimension is crucial for aligning the agent's internal state with the actual responsiveness of the GUI.

3.3 Memory Agent

To address the computational overload and logical chaos caused by directly stacking raw historical contexts, and to overcome the "loop" defect of traditional GUI agents that blindly trust instruction outputs [7, 11], we distill loose historical trajectories into an append-only state transition chain governed by mandatory validation functions.

Formally, at time step t, regarding the action a_{t-1} executed in the previous round, the Memory Agent receives the historical memory M_{t-1} and the dual frame screen captures (I_{t-1}, I_t) before and after the operation. Through the memory update function $\mathcal{M}$, it outputs the structured validated memory M_t:

$$M_t = \mathcal{M}(M_{t-1}, a_{t-1}, I_{t-1}, I_t) = \langle V_t, H_t, E_t \rangle. \tag{6}$$

The specific mechanisms and design objectives of each component are detailed as follows:

(1) Dual Frame Visual Validation (V_t): To strictly verify the real physical effects of the executed action, we introduce a dual frame visual comparison mechanism (Before/After Validation). Instead of simply recording action logs, this function compares the

visual differences on the screen before and after the execution of a_{t-1} to output a definite validation status:

$$V_t = f_{val}(a_{t-1}, I_{t-1}, I_t) \in \{\text{Success}, \text{Failure}, \text{Uncertain}\}. \quad (7)$$

An action is deemed Success only when the target element exhibits a genuine visual state change. For operations judged as Failure, the system explicitly extracts the specific elements that failed to change, thereby physically blocking the agent's action hallucination.

(2) Append-only State Transition Chain (H_t): To avoid the contextual chaos brought by lengthy raw sequences, the Memory Agent deposits only the validated state increments (ΔS_t) into the memory chain. The state chain follows a strict principle:

$$H_t = H_{t-1} \oplus \langle a_{t-1}, V_t, \Delta S_t \rangle, \quad (8)$$

where $\oplus$ denotes the sequence concatenation operation. This fact-based memory construction approach replaces raw trajectories with structured high-level summaries, providing a verified and immutable contextual baseline for the downstream planner.

(3) Anomaly Interceptor (E_t): To equip the system with the critical capabilities of active error correction and loop escape, the memory mechanism embeds a Pattern Monitor acting as a real-time anomaly interceptor. This monitor actively detects behavioral pattern degradation by comparing the current action result with the historical records in H_t:

$$E_t = \begin{cases} \text{Repeated Error}, & \text{if } a_{t-1} \in H_{t-k} \text{ and } V_t = \text{Failure} \\ \text{Stagnant State}, & \text{if } I_t \approx I_{t-n} \text{ for } n \geq N_{threshold} \\ \text{None}, & \text{otherwise.} \end{cases} \quad (9)$$

Once repetitive invalid operations on the same target (Repeated Error) or continuous physical stagnation of the interface state (Stagnant State) are detected, E_t immediately triggers hard error alerts. These alerts are directly injected into the planning prompt for the next step, forcibly interrupting the model's current behavioral inertia and compelling it to re-evaluate the state and alter its exploration strategy [21, 25, 41].

3.4 Planning Agent

The Planning Agent serves as the central cognitive hub of the system, responsible for converting high level user instructions into a coherent, executable sequence of actions. To address the computational overload and logical hallucination inherent in conventional long chain rollouts, our planner operates in a strict *step-wise mode* with memory and observation. It evaluates the current environment and accumulated facts to produce an actionable decision without needing to replay the entire raw historical trajectory.

Formally, at step t, the planning function $\mathcal{P}$ conditions on a compact tuple to generate the intermediate reasoning trace (Thought, th_t) and the explicit next action (a_t):

$$\langle th_t, A_t \rangle = \mathcal{P}(Ins, I_t, Z_t, M_{t-1}), \quad (10)$$

where Ins is the user instruction, I_t is the current raw screenshot, Z_t is the structured, unbiased semantic observation provided by the Observation Agent, and M_{t-1} is the append-only, validated state transition chain retrieved from the Memory Agent.

The planner's cognitive process is systematically decoupled into two sequential stages: **1. High level Reasoning (th_t):** Before committing to a physical operation, the planner synthesizes the multi modal inputs to infer the logical next step. It evaluates the gap between the task agnostic current state (Z_t) and the user intent (Ins). Crucially, it consults the anomaly interceptor within the memory (M_{t-1}) to ensure the proposed strategy does not repeat previously failed attempts. **2. Action Specification (a_t):** Grounded in the reasoning trace th_t, the planner selects a concrete next action $a_t \in \mathcal{A}$. The action is formulated in a structured natural language or programmatic format, which is subsequently passed to the grounding module for physical execution.

Action Space ($\mathcal{A}$): To ensure versatility across diverse GUI environments, the action space $\mathcal{A}$ encompasses two distinct paradigms. First, it includes a comprehensive suite of atomic GUI operations grounded via executable primitives (e.g., `click`, `type`, `hotkey`). Second, it incorporates a specialized `code` action, empowering the agent to generate and execute Python scripts on the fly. This dual action space allows the planner to seamlessly switch between visual spatial UI interactions and complex, system level programmatic logic. The exhaustive definition and parameters of the action space $\mathcal{A}$ are detailed in the Appendix A.

3.5 GUI Grounding Agent

The GUI Grounding Agent transforms planner decisions into executable low level interactions. It receives the planner's action A_t specification, and maps them to precise screen coordinates or element references.

Formally, the grounding agent outputs a pair:

$$a_t = (op, p) = G(A_t), \quad (11)$$

where op is a GUI primitive (click, double_click, right_click, type, hotkey, scroll) and p is either a 2D coordinate or an element ID. The grounding process unfolds in three stages:

(1) **Action parsing**. Interpret planner intent (e.g., "Click Media" → `click` operation).
(2) **Element localization**. Resolve the intended target by consulting Z_t (spatial layout + semantic roles).
(3) **Execution binding**. Translate the operation into executable code, such as `pyautogui.click()`.

After execution, the grounding agent updates the environment state by reflecting the effect of the action (e.g., the Media drop down menu opens), which is then captured in the next observation I_{t+1}. This closes the loop between perception, planning, grounding, and memory, allowing the agent to iteratively refine its trajectory while remaining robust to misalignment and redundancy.

4 Experiment

4.1 Dataset

We evaluate our method on OSWorld [37]. OSWorld is an extensible, practical computer environment designed for multimodal agents. It contains 369 tasks based on real web and desktop applications, covering file I/O, terminal operations, cross-application workflows (e.g., workflows spanning browser, email client, office suite, code editor, and system settings), multimedia processing, and automation scripting, exhibiting substantial domain diversity and

Table 1: We present a comparison of different categories of Frameworks on the OSWorld dataset. Reported values denote grounding accuracy (%) across multiple domains, including Office, Daily, Professional, Operating System (OS), Workflow (Multi-Application), and the overall average (Overall). For each domain, "Avg" denotes the average score across its sub-tasks. The final overall averages are highlighted in bold. Params (relative) indicates the approximate model composition of each framework.

Framwork	Steps	Office	Daily	Professional	OS	Workflow	Overall
General Models & Specialist Model							
O3	50	9.4	20.5	30.6	37.5	11.8	17.1
OpenCUA-32b	50	29.9	40.9	65.3	60.9	14.6	34.1
OpenCUA-72B	100	44.7	49.9	72.5	61.1	22.1	44.9
UI-TARS	100	50.4	55.6	51.0	41.6	14.6	41.8
UI-TARS-2	100	61.1	62.1	61.2	41.6	34.1	53.1
DeepMiner-Mano-7B	100	39.2	44.8	73.4	50.0	17.2	40.1
Claude-Sonnet-4.5	50	62.4	57.6	63.2	70.8	47.0	58.1
Qwen3-VL-32B-Instruct	50	–	–	–	–	–	32.4
Qwen3-VL-32B-Thinking	50	–	–	–	–	–	41.0
Agentic Framework							
Agent S3 w/ GPT-5-Mini	50	54.6	46.6	44.9	62.5	37.0	47.5
UiPath w/ GPT-5	50	49.5	62.1	71.4	73.9	37.3	53.7
Jedi-7B w/ o3	50	47.0	62.1	69.4	54.2	34.9	50.6
CoAct-1 w/ GPT-5	100	62.9	57.9	71.4	75.0	47.8	59.9
CoAct-1 w/ GPT-5	150	62.9	61.7	71.4	75.0	47.8	60.8
GTA1-7b w/ o3	50	46.1	44.9	77.6	58.3	37.1	48.6
GTA1-7b w/ o3	100	54.6	60.3	61.2	62.5	38.3	53.1
GTA1-7b w/ GPT-5	50	-	-	-	-	-	61.0
OS-SYMPHONY w/ Qwen3-VL-32B	50	40.9	53.5	75.1	58.3	31.2	46.9
OS-SYMPHONY w/ GPT5-Mini	50	58.2	61.4	75.0	73.7	47.4	58.1
OS-SYMPHONY w/ GPT-5	50	**64.9**	61.2	69.2	75.0	**54.9**	63.6
Our Methods: MGA							
MGA w/ o3	50	49.0	62.8	70.6	68.2	36.4	52.9
MGA w/ GPT-5	50	64.7	**64.4**	**85.4**	**87.5**	47.7	**64.7**

long-horizon, multi-step decision complexity. OSWorld natively supports Ubuntu operating systems and unifies cross-platform execution, multimodal input/output, and direct interaction with real third-party applications in a single evaluation framework which is unique among existing benchmarks. This makes OSWorld a standardized testbed for assessing the generalization, robustness, and scalability of general-purpose agents. The benchmark is categorized into five major domains:

4.2 Implementation Detail

All experiments were conducted in the standardized OSWorld evaluation environment. For each task, OSWorld automatically establishes the initial environment (e.g., opening specified applications or websites, downloading files to designated directories), after which we capture an initial screenshot I_0 and provide it together with the user instruction to our system and baselines. For evaluation, we adopt the rule-based evaluator provided by OSWorld. Internally, the evaluator is composed of 134 handcrafted atomic execution-based predicates, each corresponding to a verifiable outcome of the system. For a given task, the benchmark combines these atomic predicates into Boolean expressions using logical AND/OR operators. A "pass" might require conditions such as (file_exported AND MD5_matches) AND (email_sent == True).

We evaluate two variants of our proposed MGA method: 1) MGA w/ o3, both the Planner and Memory Agent are built on o3; 2) MGA w/ GPT-5, we use GPT-5 as the Planner and qwen3-8b as the Memory Agent for cost considerations, while retaining the same overall architecture and functional logic. In practice, we employ Qwen-2.5-VL-7b as the foundational vision-language model to instantiate the observation function O. We fine-tune it on the GUICourse dataset to inject GUI-domain visual grounding and UI parsing capabilities. Grounding agent adopts UI-TARS to translate the planner's natural language actions into concrete GUI operations such as clicks, inputs, or drags.

4.3 Baseline

We compare against diverse state-of-the-art methods categorized into three groups, with each method evaluated at the reasoning steps specified in the table. All comparisons are conducted on the

OSWorld dataset, with reported values representing grounding accuracy (%) across multiple domains and the overall average.

General Models & Specialist Model. This group includes general-purpose and domain-specialized models: 1) O3 [24], a cornerstone general-purpose benchmark model for open-domain reasoning (50 steps); 2) opencua-32b [32], an open-source human-computer interaction model (50 steps); 3) OpenCUA-72B, an upgraded large-scale variant (100 steps); 4) UI-TARS and UI-TARS-2, optimized for UI reasoning and control (100 steps); 5) DeepMiner-Mano-7B, a medium-scale model for targeted task execution (100 steps); 6) Claude-Sonnet-4.5 [3], featuring enhanced long-context modeling (50 steps); 7) Qwen3-VL-32B-Instruct and Qwen3-VL-32B-Thinking, multimodal models with only overall accuracy reported (50 steps).

Agentic Framework. We benchmark against prominent agentic frameworks with varied configurations: 1) Agent S3 w/ GPT-5-Mini, focused on multimodal agency (50 steps); 2) UiPath [9] w/ GPT-5, specialized in real-world automation and screen interaction (50 steps); 3) Jedi-7B w/ o3 [36], emphasizing native architectural performance (50 steps); 4) CoAct-1 w/ GPT-5 [30], tailored for multi-task coordination (100 and 150 steps); 5) GTA1-7b w/ o3 [40], a lightweight flexible execution system (50 and 100 steps); 6) GTA1-7b w/ GPT-5, an upgraded variant of GTA1-7b (50 steps); 7) OS-SYMPHONY series, with variants paired with Qwen3-VL-32B, GPT5-Mini, and GPT-5 (50 steps each).

4.4 Performance

We conduct a quantitative comparative analysis on the OSWorld dataset, comparing MGA with mainstream general models, specialist models, and SOTA agentic frameworks (see Table 1). Key results are as follows:

Operating System (OS): The OS domain (6.5% of tasks) evaluates fundamental interactions with file systems and CLI-GUI coordination. MGA w/ GPT-5 achieves 87.5% grounding accuracy within a strict 50-step limit, significantly outperforming all the SOTA model and framework. Notably, CoAct-1 requires 150 steps to reach its peak, while MGA achieves superior control with fewer interactions, demonstrating highly efficient system level trajectory planning.

Office: The Office domain (31.7%) involves complex document editing (LibreOffice) where agents must intervene in existing, high density files. This requires deep semantic perception and long-range dependency management. MGA w/ GPT-5 reaches 64.7% accuracy, matching the performance of OS-SYMPHONY (64.9%) and surpassing CoAct-1 (62.1%). Crucially, MGA achieves SOTA-level results within a strict 50-step horizon, outperforming baselines that utilize 100 or even 150 steps.

Daily: Daily tasks (21.1%) involve frequent transitions between disparate apps like Chrome, VLC, and Thunderbird. These "fragmented" scenarios often cause agents to lose context during window switching. MGA w/ GPT-5 achieves 64.4%, outperforming all baselines (avg. 62.1%). This validates robustness of MGA's observation and memory in handling heterogeneous GUI interfaces and its ability to maintain state across rapid visual changes.

Professional: The Professional domain (13.3%) tests fine-grained operations in specialized software like VS Code and GIMP, requiring spatial reasoning and domain expertise. MGA w/ GPT-5 is highly competitive at 85.4% while GTA1-7b w/ o3 leads at 77.6%, proving that MGA's architectural design effectively parses high-precision instructions without requiring massive parameter scaling.

Workflow (Multi-Application): Cross-Context Orchestration. As the most complex scenario (27.4%), the Multi-Application domain requires orchestrating data across multiple platforms (e.g., Web to Spreadsheet to Email). MGA w/ GPT-5 achieves 47.7%. Although it trails OS-Symphony (54.9%), partly due to the reasoning bottlenecks of the underlying Qwen3-8B, it still outperforms generalist agents like UiPath (37.3%) and Agent S3 (37.0%) under constrained steps. This indicates that while high-level logic remains a challenge, MGA provides a more reliable foundation for cross-app data transfer. A more granular investigation into this performance gap, is provided in Section 4.5.

Overall, MGA w/ GPT-5 achieves an average accuracy of (64.7%), establishing a new SOTA among existing methods. It outperforms OS-Symphony (63.6%) as well as other strong baselines, including GTA1-7b (61.0%) with 100 steps and CoAct-1 (60.8%) with 150 steps, setting a new Pareto frontier between accuracy and inference efficiency. These results demonstrate that our observation module explicitly captures structural and content information of documents, enabling precise semantic understanding of complex layouts. Meanwhile, the memory module maintains long-range decision context and effectively reduces trial-and-error frequency, promoting more first-time-right decisions. The average token cost and statistical overhead analysis are presented in Appendix C.

Table 2: Performance–cost trade-offs in the Workflow domain on 50 steps. The results validate that MGA achieves competitive success rates (SR) at substantially lower costs than redundant expert-ensemble frameworks. Cost calculations account for active modules under each configuration (Planner for GPT-5 and Memory for qwen3-8b; Planner and Memory for GPT-5).

Framework	Workflow SR (%)	Cost ($)
OS-Symphony	54.9	150
MGA (*w/ Qwen3 memory*)	47.7	15
MGA (*w/ GPT-5 memory*)	**56.3**	**64**

4.5 Ablation on Memory Module

To investigate the role of memory module representation capacity in long-horizon interaction tasks, we conducted controlled comparative experiments on a 50-step workflow domain by varying only the model used for the memory module while keeping the rest of the architecture unchanged. As shown in Table **??**, when using Qwen3-8B as the memory module, our framework achieved a success rate of 47.7%, which is lower than OS-Symphony that employs GPT-5 throughout its entire pipeline (54.9%). However, after replacing the memory module with GPT-5, our method's performance improved significantly to 56.3%, surpassing the baseline under the same step constraints. This controlled substitution reveals that performance differences do not primarily stem from architectural complexity, but from the quality of memory state abstraction during long-horizon interactions.

Qwen3-8B exhibits representational capacity limitations in long-horizon intent retention, critical state milestone extraction, and implicit error tracing. These limitations lead to information compression loss in trajectory summarization and semantic drift of task intent as interaction steps increase, making it difficult to maintain execution robustness comparable to OS-Symphony in 50-step long-horizon sequences.

After upgrading the memory module to GPT-5, our framework, with only a single GPT-5 memory unit and a single GPT-5 planner, surpassed OS-Symphony which relies on the collaboration of six GPT-5 modules. This performance reversal reveals the following core insights: 1) High-quality state abstraction and trajectory summarization can implicitly compensate for the explicit scheduling and verification overhead in multi-agent systems, maintaining context consistency without additional verification modules. 2) In computer-use agents, the impact of long-term memory fidelity on final decision-making performance is significantly greater than the complexity of modular engineering design. 3) A minimalist architecture combined with high-capacity memory models can achieve superior long-horizon task planning and execution robustness with significantly reduced inference overhead and system complexity. Our framework retains only a minimal viable pipeline: a visual perception module (Observer, Qwen-VL-7B), a decision-making module (Planner, GPT-5), and a memory abstraction module (Memory), explicitly removing auxiliary scheduling, multi-module state verification, and complex coordination mechanisms. In the initial configuration, only the planner uses a high-capacity model. Detailed case studies on the iterative transformation of memory are provided in Appendix B.

4.5.1 Performance-CostTrade-offs. Notably, table 2 reports the success rates and operational costs under different model configurations.

Efficiency Frontier: When utilizing GPT-5 as the memory model, MGA achieves a 56.3% success rate at a cost of $64 per task. In comparison, OS-Symphony requires an expenditure of $150 to reach a 54.9% success rate. The data suggests that MGA's coordinated "observation-memory" logic is more efficient for practical tasks than frameworks built on redundant expert model stacks.

Cost Elasticity: MGA demonstrates significant cost elasticity. By switching to a lightweight memory model (Qwen3), the cost per task drops to $15—approximately 10% of the OS-Symphony's cost. Even in this high-efficiency configuration, MGA retains a success rate of 47.7%, validating the framework's viability for large-scale deployment or resource-constrained environments.

4.6 Ablation on Core Components

To verify the necessity and synergistic effects of MGA's core components, we conduct ablations on the Workflow domain, with results summarized in Table 3.

Memory Module. Removing the memory module (**w/o memory**, emptying $\mathcal{M}_{t-1}$) causes a sharp accuracy drop to 39.0% (GPT-5) and 27.7% (O3). This confirms the memory module's core role: it distills historical trajectories to avoid context bloat and state forgetting, which is critical for long-horizon tasks.

Observation-First Mechanism. Eliminating the structured observation Z_t (**w/o obs**) reduces accuracy to 46.0% (GPT-5) and 35.1% (O3). The observation module mitigates action bias by extracting complete semantic content and topology, thereby reducing interference from transient visual noise.

Table 3: Ablation study on core components. w/o obs removes the structured observation Z_t; w/o memory empties the distilled memory $\mathcal{M}_{t-1}$ (reverting to plain trajectory execution). Reported values are grounding accuracy (%) across domains.

Configuration	GPT-5	O3
MGA (Full)	56.3	36.4
- w/o obs	46.0	35.1
- w/o memory	39.0	27.7
- w/o obs + w/o memory	36.8	15.8

Synergistic Effects. Removing both components yields the worst performance (36.8% for GPT-5, 15.8% for O3). The significant drop demonstrates that Z_t and $\mathcal{M}_{t-1}$ are highly synergistic: perception filters noise, and memory condenses semantics, together forming a robust state-tracking pipeline.

5 Conclusion

We introduce MGA, a lightweight agent paradigm that rethinks interaction pipelines by replacing monolithic context windows and bloated collaboration protocols with decoupled decision nodes anchored by a structured memory chain. The Observer First mechanism ensures unbiased, high-fidelity state perception by strictly separating factual screen reading from intent-driven planning, thereby mitigating visual hallucinations and action bias. Complementarily, the Structured Memory mechanism replaces verbose historical logs with an append-only, validation-gated state transition chain. By enforcing dual-frame visual comparison after each action, the system effectively intercepts error cascades, prevents behavioral stagnation, and enables proactive online correction. Empirical evaluations on OSWorld and diverse real-world applications confirm that MGA delivers competitive task success rates without relying on heavyweight toolsets or extensive context management. Our results demonstrate that architectural minimalism, grounded state tracking, and disciplined perception are sufficient to unlock reliable long-horizon GUI automation. While MGA significantly reduces cognitive load and inference latency, its reliance on baseline MLLM capabilities for complex visual grounding presents a natural boundary. Future work will explore adaptive memory compression strategies, cross-application state transfer, and tighter integration with low-level execution engines to further enhance robustness in highly dynamic, open-world environments. Ultimately, MGA provides a principled, efficient alternative to bloated agent frameworks, paving the way for more scalable, deployable, and human-aligned GUI automation systems.

References

[1] Agashe, S., Han, J., Gan, S., Yang, J., Li, A., Wang, X.E.: Agent s: An open agentic framework that uses computers like a human. arXiv preprint arXiv:2410.08164 (2024)
[2] Agashe, S., Wong, K., Tu, V., Yang, J., Li, A., Wang, X.E.: Agent s2: A compositional generalist-specialist framework for computer use agents. arXiv preprint arXiv:2504.00906 (2025)
[3] Anthropic, C.: 3.7 sonnet and claude code (2025)

[4] Bai, S., Cai, Y., Chen, R., Chen, K., Chen, X., Cheng, Z., Deng, L., Ding, W., Gao, C., Ge, C., et al.: Qwen3-vl technical report. arXiv preprint arXiv:2511.21631 (2025)

[5] Bai, S., Chen, K., Liu, X., Wang, J., Ge, W., Song, S., Dang, K., Wang, P., Wang, S., Tang, J., et al.: Qwen2. 5-vl technical report. arXiv preprint arXiv:2502.13923 (2025)

[6] Caffagni, D., Cocchi, F., Barsellotti, L., Moratelli, N., Sarto, S., Baraldi, L., Cornia, M., Cucchiara, R.: The revolution of multimodal large language models: a survey. arXiv preprint arXiv:2402.12451 (2024)

[7] Chen, M., Li, Y., Yang, Y., Yu, S., Lin, B., He, X.: Automanual: Constructing instruction manuals by llm agents via interactive environmental learning. Advances in Neural Information Processing Systems **37**, 589–631 (2024)

[8] Cheng, K., Sun, Q., Chu, Y., Xu, F., YanTao, L., Zhang, J., Wu, Z.: Seeclick: Harnessing gui grounding for advanced visual gui agents. In: Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers). pp. 9313–9332 (2024)

[9] Cristescu, H., Park, C., Nguyen, T.C., Talmacel, S., Ilie, A.G., Adam, S.: Ui-cube: Enterprise-grade computer use agent benchmarking beyond task accuracy to operational reliability. arXiv preprint arXiv:2511.17131 (2025)

[10] Dong, L., Zhou, Z., Yang, S., Sheng, H., Cheng, P., Wu, Z., Wu, Z., Liu, G., Zhang, Z.: Say one thing, do another? diagnosing reasoning-execution gaps in vlm-powered mobile-use agents. arXiv preprint arXiv:2510.02204 (2025)

[11] Favreau, P.L., Lo, J.P., Guiguet, C., Simon-Meunier, C., Dehandschoewercker, N., Roush, A.G., Goldfeder, J., Shwartz-Ziv, R.: Do multi-agents dream of electric screens? achieving perfect accuracy on androidworld through task decomposition. arXiv preprint arXiv:2602.07787 (2026)

[12] Fu, T., Su, A., Zhao, C., Wang, H., Wu, M., Yu, Z., Hu, F., Shi, M., Dong, W., Wang, J., et al.: Mano technical report. arXiv preprint arXiv:2509.17336 (2025)

[13] Gonzalez-Pumariega, G., Tu, V., Lee, C.L., Yang, J., Li, A., Wang, X.E.: The unreasonable effectiveness of scaling agents for computer use. arXiv preprint arXiv:2510.02250 (2025)

[14] Han, W., Zeng, Z., Huang, J., Jiang, S., Zheng, L., Yang, L., Qiu, H., Yao, C., Chen, J., Ma, L.: Guirobotron-speech: Towards automated gui agents based on speech instructions. arXiv preprint arXiv:2506.11127 (2025)

[15] Hong, W., Wang, W., Lv, Q., Xu, J., Yu, W., Ji, J., Wang, Y., Wang, Z., Dong, Y., Ding, M., et al.: Cogagent: A visual language model for gui agents. In: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition. pp. 14281–14290 (2024)

[16] Huang, J., Zeng, Z., Han, W., Zhong, Y., Zheng, L., Fu, S., Chen, J., Ma, L.: Scaletrack: Scaling and back-tracking automated gui agents. arXiv preprint arXiv:2505.00416 (2025)

[17] Jiang, Y., Zheng, Y., Wan, Y., Han, J., Wang, Q., Lyu, M.R., Yue, X.: Screencoder: Advancing visual-to-code generation for front-end automation via modular multimodal agents. arXiv preprint arXiv:2507.22827 (2025)

[18] Li, K., Meng, Z., Lin, H., Luo, Z., Tian, Y., Ma, J., Huang, Z., Chua, T.S.: Screenspot-pro: Gui grounding for professional high-resolution computer use. arXiv preprint arXiv:2504.07981 (2025)

[19] Li, T., Li, G., Deng, Z., Wang, B., Li, Y.: A zero-shot language agent for computer control with structured reflection. arXiv preprint arXiv:2310.08740 (2023)

[20] Liu, Y., Li, P., Wei, Z., Xie, C., Hu, X., Xu, X., Zhang, S., Han, X., Yang, H., Wu, F.: Infiguiagent: A multimodal generalist gui agent with native reasoning and reflection. arXiv preprint arXiv:2501.04575 (2025)

[21] Madaan, A., Tandon, N., Gupta, P., Hallinan, S., Gao, L., Wiegreffe, S., Alon, U., Dziri, N., Prabhumoye, S., Yang, Y., et al.: Self-refine: Iterative refinement with self-feedback. Advances in Neural Information Processing Systems **36**, 46534–46594 (2023)

[22] Nguyen, D., Chen, J., Wang, Y., Wu, G., Park, N., Hu, Z., Lyu, H., Wu, J., Aponte, R., Xia, Y., Li, X., Shi, J., Chen, H., Lai, V.D., Xie, Z., Kim, S., Zhang, R., Yu, T., Tanjim, M., Ahmed, N.K., Mathur, P., Yoon, S., Yao, L., Kveton, B., Kil, J., Nguyen, T.H., Bui, T., Zhou, T., Rossi, R.A., Dernoncourt, F.: Gui agents: A survey. arXiv preprint arXiv:2412.13501 (2024). https://doi.org/10.48550/arXiv.2412.13501, https://arxiv.org/abs/2412.13501, submitted 18 Dec 2024; Revised 26 Sep 2025

[23] Nguyen, D., Chen, J., Wang, Y., Wu, G., Park, N., Hu, Z., Lyu, H., Wu, J., Aponte, R., Xia, Y., et al.: Gui agents: A survey. In: Findings of the Association for Computational Linguistics: ACL 2025. pp. 22522–22538 (2025)

[24] OpenAI, T.: Introducing openai o3 and o4-mini. https://openai.com/index/introducing-o3-and-o4-mini/ (2025)

[25] Packer, C., Fang, V., Patil, S., Lin, K., Wooders, S., Gonzalez, J.: Memgpt: Towards llms as operating systems. (2023)

[26] Qin, Y., Ye, Y., Fang, J., Wang, H., Liang, S., Tian, S., Zhang, J., Li, J., Li, Y., Huang, S., et al.: Ui-tars: Pioneering automated gui interaction with native agents. arXiv preprint arXiv:2501.12326 (2025)

[27] Qiu, J., Qi, X., Zhang, T., Juan, X., Guo, J., Lu, Y., Wang, Y., Yao, Z., Ren, Q., Jiang, X., et al.: Alita: Generalist agent enabling scalable agentic reasoning with minimal predefinition and maximal self-evolution. arXiv preprint arXiv:2505.20286 (2025)

[28] Shaw, P., Joshi, M., Cohan, J., Berant, J., Pasupat, P., Hu, H., Khandelwal, U., Lee, K., Toutanova, K.N.: From pixels to ui actions: Learning to follow instructions via graphical user interfaces. Advances in Neural Information Processing Systems **36**, 34354–34370 (2023)

[29] Shinn, N., Cassano, F., Gopinath, A., Narasimhan, K., Yao, S.: Reflexion: Language agents with verbal reinforcement learning. Advances in Neural Information Processing Systems **36**, 8634–8652 (2023)

[30] Song, L., Dai, Y., Prabhu, V., Zhang, J., Shi, T., Li, L., Li, J., Savarese, S., Chen, Z., Zhao, J., et al.: Coact-1: Computer-using agents with coding as actions. arXiv preprint arXiv:2508.03923 (2025)

[31] Wang, S., Liu, W., Chen, J., Zhou, Y., Gan, W., Zeng, X., Che, Y., Yu, S., Hao, X., Shao, K., et al.: Gui agents with foundation models: A comprehensive survey. arXiv preprint arXiv:2411.04890 (2024)

[32] Wang, X., Wang, B., Lu, D., Yang, J., Xie, T., Wang, J., Deng, J., Guo, X., Xu, Y., Wu, C.H., et al.: Opencua: Open foundations for computer-use agents. arXiv preprint arXiv:2508.09123 (2025)

[33] Wanyan, Y., Zhang, X., Xu, H., Liu, H., Wang, J., Ye, J., Kou, Y., Yan, M., Huang, F., Yang, X., et al.: Look before you leap: A gui-critic-r1 model for pre-operative error diagnosis in gui automation. arXiv preprint arXiv:2506.04614 (2025)

[34] Wu, Q., Bansal, G., Zhang, J., Wu, Y., Li, B., Zhu, E., Jiang, L., Zhang, X., Zhang, S., Liu, J., et al.: Autogen: Enabling next-gen llm applications via multi-agent conversations. In: First Conference on Language Modeling (2024)

[35] Wu, Z., Wu, Z., Xu, F., Wang, Y., Sun, Q., Jia, C., Cheng, K., Ding, Z., Chen, L., Liang, P.P., et al.: Os-atlas: A foundation action model for generalist gui agents. arXiv preprint arXiv:2410.23218 (2024)

[36] Xie, T., Deng, J., Li, X., Yang, J., Wu, H., Chen, J., Hu, W., Wang, X., Xu, Y., Wang, Z., et al.: Scaling computer-use grounding via user interface decomposition and synthesis. arXiv preprint arXiv:2505.13227 (2025)

[37] Xie, T., Zhang, D., Chen, J., Li, X., Zhao, S., Cao, R., Hua, T.J., Cheng, Z., Shin, D., Lei, F., et al.: Osworld: Benchmarking multimodal agents for open-ended tasks in real computer environments. Advances in Neural Information Processing Systems **37**, 52040–52094 (2024)

[38] Xu, Y., Wang, Z., Wang, J., Lu, D., Xie, T., Saha, A., Sahoo, D., Yu, T., Xiong, C.: Aguvis: Unified pure vision agents for autonomous gui interaction. arXiv preprint arXiv:2412.04454 (2024)

[39] Yang, B., Jin, K., Wu, Z., Liu, Z., Sun, Q., Li, Z., Xie, J., Liu, Z., Xu, F., Cheng, K., Li, Q., Wang, Y., Qiao, Y., Wang, Z., Ding, Z.: Os-symphony: A holistic framework for robust and generalist computer-using agent (2026), https://arxiv.org/abs/2601.07779

[40] Yang, Y., Li, D., Dai, Y., Yang, Y., Luo, Z., Zhao, Z., Hu, Z., Huang, J., Saha, A., Chen, Z., et al.: Gta1: Gui test-time scaling agent. arXiv preprint arXiv:2507.05791 (2025)

[41] Yao, S., Zhao, J., Yu, D., Du, N., Shafran, I., Narasimhan, K., Cao, Y.: React: Synergizing reasoning and acting in language models. In: International Conference on Learning Representations (ICLR) (2023)

[42] Ye, J., Zhang, X., Xu, H., Liu, H., Wang, J., Zhu, Z., Zheng, Z., Gao, F., Cao, J., Lu, Z., et al.: Mobile-agent-v3: Foundamental agents for gui automation. arXiv preprint arXiv:2508.15144 (2025)

[43] Yin, S., Fu, C., Zhao, S., Li, K., Sun, X., Xu, T., Chen, E.: A survey on multimodal large language models. National Science Review **11**(12), nwae403 (2024)

[44] Zhai, S., Bai, H., Lin, Z., Pan, J., Tong, P., Zhou, Y., Suhr, A., Xie, S., LeCun, Y., Ma, Y., et al.: Fine-tuning large vision-language models as decision-making agents via reinforcement learning. Advances in neural information processing systems **37**, 110935–110971 (2024)

[45] Zhang, C., He, S., Qian, J., Li, B., Li, L., Qin, S., Kang, Y., Ma, M., Liu, G., Lin, Q., et al.: Large language model-brained gui agents: A survey. arXiv preprint arXiv:2411.18279 (2024)

[46] Zhang, C., Huang, H., Ni, C., Mu, J., Qin, S., He, S., Wang, L., Yang, F., Zhao, P., Du, C., et al.: Ufo2: The desktop agentos. arXiv preprint arXiv:2504.14603 (2025)

[47] Zhao, S., Zhang, H., Lin, S., Li, M., Wu, Q., Zhang, K., Wei, C.: Pyvision: Agentic vision with dynamic tooling. arXiv preprint arXiv:2507.07998 (2025)

A Action Abstraction and Parameterized Schema

A core design challenge in GUI agents is defining an action abstraction that remains robust across long-horizon task execution, varying screen layouts, and ambiguous interface states. As shown in Table 4, MGA adopts a discrete, parameterized action space $\mathcal{A}$ that combines semantic GUI interactions with programmatic execution through a dedicated `code` action to enable robust desktop automation. We represent each action as a parameterized tuple $a_t = (op, p)$, where op denotes the action type and $p$denotes its associated arguments.

Semantic-grounded GUI Actions. Following prior work such as [40], the GUI action set includes atomic interaction primitives such as `click`, `drag_and_drop`, `type`, `scroll`, and `hotkey`. Each action is parameterized by a semantic target (e.g., a natural language description of a UI element) and optional action-specific arguments (e.g., input text, scroll direction, key combinations).

Instead of relying on fixed coordinates, MGA resolves interaction targets through a Visual Grounding Model at execution time. Given a semantic description, the grounding module predicts the corresponding screen location, enabling robustness to layout variation and resolution changes. Variants of actions (e.g., single vs. double click) are expressed through parameters, ensuring that each action remains atomic.

Programmatic Execution via `code`. In addition to GUI actions, the action space includes a `code` action that allows the agent to generate and execute Python scripts, with execution results returned to the environment. This action is parameterized by an executable code snippet, which operates directly on system-level resources (e.g., files, structured data). It provides a complementary interaction modality when GUI-based manipulation is inefficient or imprecise, such as in dense interfaces (e.g., spreadsheets) or background operations.

Control and Termination Signals. To support stable long-horizon execution, we introduce a small set of control signals. The `wait` action pauses execution to accommodate asynchronous system responses. The `done` and `fail` signals indicate successful completion and unsuccessful termination, respectively. These signals are not executable interactions but are included to explicitly manage task flow during rollout.

B Case Study: Dynamic Evolution of Agent Memory

To demonstrate the efficacy of our memory module in long-horizon tasks, we present a detailed case study of the agent executing a multi-document information extraction and formatting task.

Task Description: The agent is instructed to open the "Grammar test" folder, extract the multiple-choice answers for Test 2 and Test 3, and write them into an `Answer.docx` file, strictly following the formatting of Test 1 (e.g., "`bbbad`").

Over the course of 33 steps, the agent's memory module dynamically evolves, proving crucial for error recovery, cross-context reasoning, and infinite loop avoidance. The evolution can be divided into four distinct phases (Table. 5):

Phase 1: Environmental Exploration and Knowledge Correction (Steps 1–8)

Initially, based on the user's prompt, the agent assumes the target files are inside the "Grammar test" directory on the desktop. Upon opening the folder (Step 1), the visual observation reveals a "Folder is Empty" message. At this juncture, the *Latest Action Verification* flags the unexpected state, prompting an immediate update to the *Cumulative Knowledge Base*: *"The 'Grammar test' folder is empty; the relevant files are on the Desktop."* This dynamic correction prevents the agent from hallucinating or endlessly searching within the empty directory, seamlessly redirecting its attention to the desktop files.

Phase 2: Cross-Document Working Memory (Steps 9–23)

In this phase, the agent must retain the target format from `Answer.docx` while navigating other documents. During the exploration of `Grammar test 2.docx` and `Grammar test 3.docx`, the agent utilizes scrolling actions to gather fragmented visual information (individual test questions). The *Cumulative Knowledge Base* acts as a working memory, successfully abstracting raw visual inputs into condensed logical strings (e.g., logging *"Derived Test 2 answer string: 'baaad'"*). This prevents the agent from forgetting early answers while reading subsequent pages.

Phase 3: Anti-Stagnation Recovery (Steps 24–27)

A critical bottleneck occurs when the agent attempts to insert the derived answers into `Answer.docx`. The agent tries to place the cursor on an empty line below the "Grammar test 3:" label using a mouse click (Step 24, Figure 3a) and the `Enter` key (Step 25, Figure 3b), both of which fail to produce a visible line break.

At Step 26 (Figure 3c), *Repetition & Stagnation Detection* memory is triggered, throwing a critical warning: `STAGNANT: GUI state has persisted for 2 consecutive steps.`

Triggered by this explicit warning, the agent aborts the failing trajectory. It reasons that the cursor position is ambiguous and autonomously falls back to a more robust strategy: pressing the `End` key to ensure the cursor is at the absolute end of the line before pressing `Enter` again. This self-healing capability breaks the deadlock without human intervention, and thus finally make the phase success in step 27(Figure 3d).

Phase 4: Goal Prioritization and Infinite Loop Avoidance (Steps 28–33)

In the final phase, the agent attempts to save the document using `Ctrl+S` (Step 28) and the UI save button (Step 29). However, due to LibreOffice's silent saving mechanism, the *Latest Action Verification* returns an `UNCERTAIN` state, as no visual confirmation dialog appears.

By Step 33, the memory module flags a `REPEATED` warning for the save hotkey. Instead of getting stuck in an infinite loop of attempting to verify the save, the agent consults its *Cumulative Knowledge Base*, verifying that the core objective (writing "baaad" and "aaaaa" in the correct format) is visually confirmed. Prioritizing the global task completion over local sub-step uncertainty, the agent gracefully terminates the process by calling `agent.done()`.

C System-Level Resource and Inference Analysis

In this section, we provide a quantitative evaluation of the MGA framework's efficiency. By analyzing inference overhead, modular resource distribution, and operational costs, we demonstrate

Table 4: Agent action space with parameterized action schema.

Action	Parameters	Description	General Constraints
		Semantic-grounded GUI Actions	

click	`instruction: str`
	`num_clicks: int`
	`button_type: str`
	`hold_keys: list`
		Click on a semantically described UI element.	Target elements are specified via semantic descriptions.

drag_and_drop	`starting_desc: str`
	`ending_desc: str`
	`hold_keys: list`
		Drag from one semantically described element to another.	Both source and target are resolved via semantic grounding.

type	`element_desc: str`
	`text: str`
	`overwrite: bool`
	`enter: bool`
		Type text into the specified UI element.	Supports optional overwrite and submission behavior.

hotkey	`keys: list`	Execute a keyboard shortcut.	Keys are provided as standard key combinations.

scroll	`instruction: str`
	`clicks: int`
	`shift: bool`
		Scroll within a semantically specified region.	Scroll direction and magnitude are parameterized.

		Programmatic Execution via `code`	
code	`code: str`	Execute a Python script for direct system-level operations.	Enables precise manipulation of structured data and background resources.
		Control and Termination Signals	
wait	`time: float`	Pause execution for a specified duration.	Handles asynchronous UI responses and latency.
done	`return_value: Optional`	Signal successful task completion.	Terminates execution and optionally returns results.
fail	None	Signal unsuccessful termination.	Used when the task cannot be completed under the current trajectory.

how our first-principles design addresses the dual bottlenecks of historical interference and system redundancy.

C.1 Inference Efficiency and Context Management

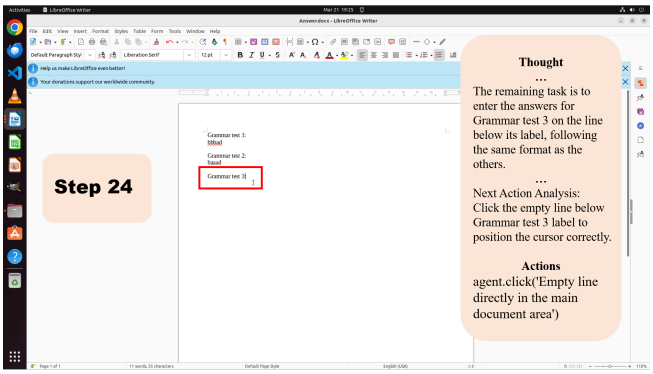
Table 6 compares the inference overhead of MGA against OS-Symphony. In contrast to mainstream frameworks that rely on an ensemble of expert agents to manage complexity, MGA utilizes a streamlined single-step execution logic.

Exploration Cost and Global Efficiency: Experimental results indicate that MGA's average execution step count (19.3) is higher than the OS-Symphony (15.2). Admittedly, this minimalist design may require a higher number of exploration steps to complete certain complex tasks. However, due to MGA's efficient distillation and compression of interaction history, its prompt token consumption per step is significantly lower than that of the OS-Symphony. Even with a higher step count, MGA's total token consumption (226.1K) is only approximately 38% of the OS-Symphony's (598.6K), demonstrating the global economic advantage of a low-cost per-step design.

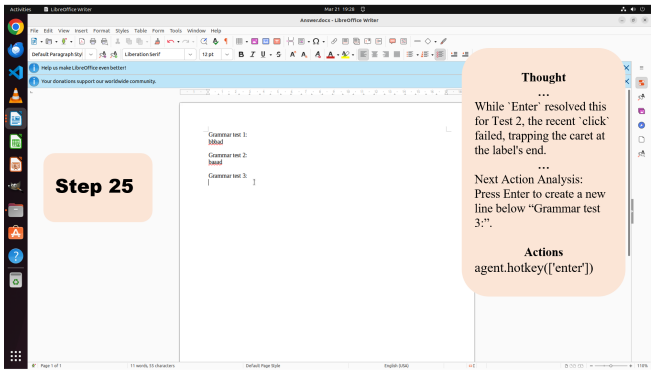
C.2 Modular Design Rationale

Figure 4 illustrates the distribution of token consumption across MGA's modules. As a Memory-driven Agent, the Memory module accounts for the largest proportion of token overhead (42.6%), as it performs the computationally intensive tasks of state validation and history compression.

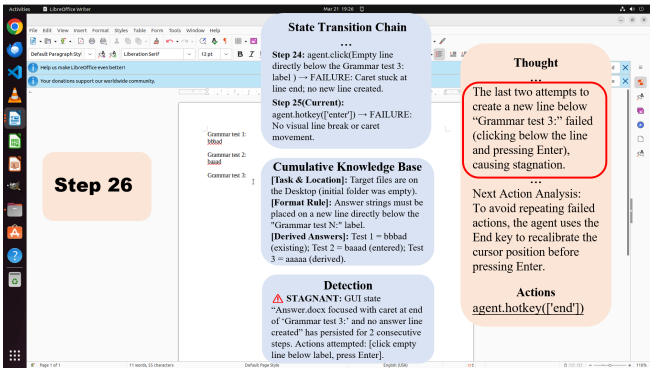
Indispensability of the Observer Module: The Observer module's consumption remains stable at 45.6K tokens. This module is not designed for cost reduction; rather, it is an essential component that provides a significantly higher level of perceptual granularity than a standalone Planner. While the Observer introduces higher-dimensional state descriptions, the modular architecture ensures that this added detail does not compromise overall economy. MGA's



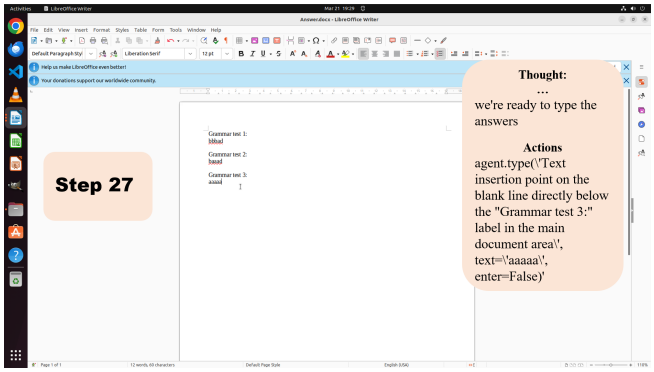
(a) Step 24: Ready to input answer



(b) Step 25: Stagnation (Enter key fails)



(c) Step 26: Memory triggers replanning trajectory



(d) Step 27: Recovery (Successfully typed 'aaaaa')

Figure 3: Visualizing the Anti-Stagnation Recovery process. (a) The agent prepares to input the answer. (b) When basic operations fail to create a new line due to UI ambiguity, a stagnation occurs. (c) The memory module detects the lack of visual state change and triggers a replanning trajectory. (d) The agent autonomously adapts by injecting an `End` hotkey to recalibrate the cursor, successfully completing the text insertion.

Table 5: Timeline of memory evolution and agent strategy adjustment.

Phase	Steps	Memory State Update	Consequent Agent Action
1. Knowledge Correction	1–8	Action verification detects "Folder is empty", correcting the knowledge base: files reside on the Desktop rather than the target folder.	Aborts folder search and redirects grounding toward Desktop icons via the open dialog.
2. Working Memory	9–23	Aggregates fragmented options from scrolling interactions and stores derived answers (e.g., Test 2 = "baaad", Test 3 = "aaaaa").	Returns to Answer.docx using pre-computed semantic strings, avoiding redundant copy-paste operations.
3. Anti-Stagnation	24–27	Stagnation detection triggers after repeated `click` and `enter` fail to produce visible changes.	Terminates ineffective retries and applies a fallback strategy (`End` hotkey) to reset cursor position.
4. Loop Avoidance	28–33	Action verification returns `UNCERTAIN` due to lack of visual feedback during save operations and flags repeated hotkey usage.	Prioritizes global task completion over local uncertainty and safely terminates via `done()`.

total operational cost remains substantially lower than that of the OS-Symphony.

Table 6: Inference efficiency on OSWorld. MGA's minimalist single-step logic results in a higher average step count for exploration; however, due to low per-step costs achieved through history distillation, total token consumption remains significantly lower than the OS-Symphony. Token statistics (in thousands, K) include prompt, completion, and total tokens averaged across all task categories.

Framework	Avg. Step	Prompt Tokens	Completion Tokens	Total
OS-Symphony	15.2	572.9K	25.7K	598.6K
MGA	19.3	188.8K	37.3K	226.1K

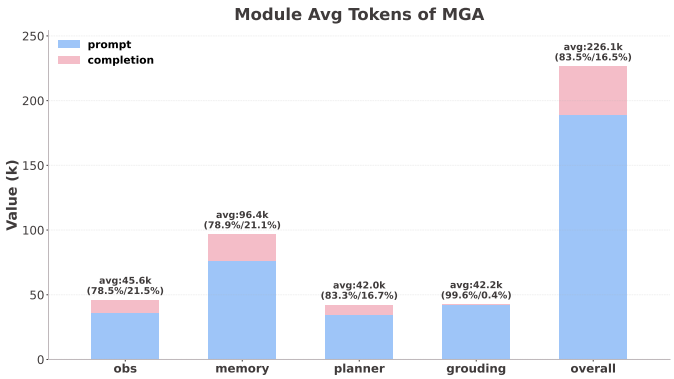


Figure 4: Token distribution across MGA modules. The dominant overhead of the Memory module (42.6%) reflects its core role in the memory-driven architecture, while the Observer module provides essential perceptual granularity. Stacked bars represent average prompt and completion tokens (in thousands) for each module, with relative percentages indicating their respective proportions of total system overhead.

D Prompt Design

To ensure modular synergy and minimize context redundancy, we developed a set of role-specific system prompts based on the principle of functional isolation.

Memory agent: The Memory agent (Fig. 5) is tasked with maintaining the *State Transition Chain*. Unlike naive history concatenation, our prompt enforces an **Append-Only** rule and a **Verification Logic Rule**. This forces the model to distill unprocessed interaction logs into verified state increments, which is the key driver behind the inference efficiency discussed in Section C.

Observer: The Observer's prompt (Fig. 6) is strictly constrained to report only visible GUI states. By explicitly prohibiting intent inference ("DO NOT INFER INTENT"), we decouple raw perception from subjective planning, thereby mitigating confirmation bias in complex UI environments.

Planner: The Planner (Fig. 7) operates on a coordinate-free Python API. By mandating functional element descriptions (e.g., "the Search bar") over numeric pixels, the prompt shifts the Planner's focus from low-level spatial grounding to high-level strategic reasoning. Furthermore, the mandatory `Thought-Action` template ensures that each decision is grounded in the distilled history provided by the Memory module.

Received 20 February 2007; revised 12 March 2009; accepted 5 June 2009

Figure 5: MEMORY AGENT SYSTEM PROMPT

MEMORY AGENT SYSTEM PROMPT

1. STATE TRANSITION CHAIN

- **Narrative**: Single coherent chain.
- **Append-Only**: Lock previous history; append only latest verified delta.
- **Delta**: Explicitly record failures or "no visible effect" steps.

2. CUMULATIVE KNOWLEDGE

- **Extraction**: Save specific IDs, document rules, or key requirements.
- **Goal Tracking**: Record high-level goal updates discovered in GUI.
- **Filter**: Discard UI noise; keep task-relevant static facts.

3. VERIFICATION LOGIC RULE

- **Visual Proof**: Compare Image 1 (Before) vs Image 2 (After).
- **Success**: `click` (state change), `type` (text visible), `code` (no crash).
- **Anomaly**: Flag repeated actions or stagnant GUI states ($\geq$ 2 steps).

MANDATORY OUTPUT FORMAT (STRICT COMPLIANCE)

```
1. State Transition Chain
[History Narrative: Step 1 ... Step N-1]
[Current Step Narrative: Step N(Current): `action` → Outcome]

2. Cumulative Knowledge Base
Knowledge & Constraints: [Fact/Instruction 1], [Fact/Instruction 2]

3. Latest Action Verification
Result: [SUCCESS / FAILURE / UNCERTAIN]
Evidence: [Specific visual changes between Image 1 & 2]

4. Repetition & Stagnation Detection
[Repetition Check] | [Stagnation Check]
```

STRICT CONSTRAINTS: NO STRATEGIC ADVICE. NO RECOMMENDATIONS. NO INFERENCE BEYOND VISUALS. APPEND-ONLY VERIFIED FACTS.

Figure 6: OBSERVER SYSTEM PROMPT

OBSERVER SYSTEM PROMPT

1. WINDOW & APP UI STATE

- **Foreground**: Identify the focused window receiving input.
- **Hierarchy**: List all titles, overlapping windows, and taskbar indicators.
- **Modals**: Explicitly describe blocking dialogs, alerts, or file pickers.

2. VISIBLE TEXT CONTENT

- **Verbatim**: Transcribe terminal output, editor text, and cell values.
- **UI Labels**: Record menu items, tab names, and sidebar entries.
- **Forms**: Report current values or placeholders in input/search bars.

3. INTERACTIVE UI STATE

- **Focus**: Locate the blinking caret, highlighted border, or selection.
- **Progress**: Note spinners, loading bars, or "Processing..." text.
- **Overlays**: Detect tooltips, context menus, or autocomplete lists.

MANDATORY OBSERVATION OUTPUT FORMAT

```
1. Application & Window State
[Foreground Window | Background Windows | System Dialogs]

2. Visible Text Content
[Document/Editor Content | UI Labels | Notifications | Form Values]

3. Current Interactive State
[Cursor/Focus | Selection | Scroll Position | Loading State | Overlays]
```

STRICT CONSTRAINTS: DO NOT SUGGEST NEXT STEPS. DO NOT INFER INTENT. REPORT ONLY WHAT IS DIRECTLY VISIBLE.

Figure 7: PLANNER SYSTEM PROMPT

PLANNER SYSTEM PROMPT

AVAILABLE ACTIONS (Python API)

```
click(desc, n, type, keys)      Click described element. n=2 (double) for opening files; n=3 for paragraphs.
type(desc, text, ovr, ent)      Type text. overwrite=True clears field; enter=True submits.
drag_and_drop(start, end)       Drag from start description to end description. No coordinates.
code(description)               Execute natural language logic as Python scripts (Calculations/Excel batch).
hotkey(keys) | scroll(desc, n)  Execute combinations (e.g., ['ctrl', 's']) or scroll within elements.
done() | fail()                 Signal task completion or trigger replanning on failure/stagnation.
```

1. ELEMENT DESCRIPTION RULES

- **No Coordinates**: Never output pixels; use visual/functional descriptions.
- **No Special Symbols**: Avoid `'Sheet1'`, use *Sheet1 tab at bottom.*
- **One Step**: Perform only one verifiable UI action per turn.

2. OPERATIONAL LOGIC RULE

- **Verification**: Reopen apps to verify file changes made via code.
- **Recovery**: If Memory reports "REPEATED", call `agent.fail()` to replan.
- **Success**: Use `agent.done()` only when the task is fully verified.

3. DIRECT CODE CONTROL USAGE

- **Use for**: Batch data entry, precise cell (D2) targeting, or calculations.
- **Prohibited**: No PyAutoGUI, no charts, no visual-only elements.
- **Results**: Raise Exception with `"RESULT: ..."` to return values.

MANDATORY OUTPUT FORMAT

```
Thought:
- Step by Step Progress Assessment (Analyze memory/history)
- Next Action Analysis (Evaluate options & consequences)
Action:
agent.click("Save button in the top toolbar")
Scripts: (Only for agent.code() actions)
[Raw executable Python code]
```

STRICT CONSTRAINT

NEVER USE NUMERIC COORDINATES.
NEVER REPEAT FAILED STRATEGIES.
OVERWRITE FILES BY DEFAULT.